



آزمایشگاه طراحی سیستم‌های دیجیتال

ترم تابستان ۱۴۰۵

استاد: دکتر انصاری

آزمایش دوم کنترل کننده‌ی اتاق انتظار (شمارنده‌ی بالا/پایین + قفل ورود) معین آعلی

خلاصه

در این آزمایش یک مدار ترتیبی (دارای فلیپ‌فلاپ و کلاک) در Logisim Evolution طراحی شده که ورود و خروج افراد به یک اتاق با ظرفیت محدود (حداکثر ۱۵ نفر) را کنترل می‌کند. مدار waiting_room از دو زیرسیستم تشکیل شده: یک شمارنده‌ی بالا/پایین ۴ بیتی که تعداد افراد داخل اتاق را نگه می‌دارد (با پالس IN افزایش، با پالس OUT کاهش می‌یابد)، و یک قفل درخواست ورود که با ترکیب سیگنال‌های Ent (دکمه‌ی درخواست ورود)، T (در بازه‌ی مجاز زمانی) و پرشدن یا نشدن اتاق، خروجی Open (فرمان باز شدن در) را تولید می‌کند. برخلاف آزمایش اول، اینجا هیچ قطعه‌ی حاضر و آماده‌ی Counter از کتابخانه‌ی Memory استفاده نشده — شمارنده‌ی بالا/پایین کاملاً از پایین به بالا و با ۴ قطعه‌ی Register تک‌بیتی و منطق toggle ساخته شده از گیت‌های NOT/AND/XOR پیاده شده، دقیقاً به همان روحیه‌ی «ساخت از بلوک‌های پایه» که در آزمایش اول دنبال شد.

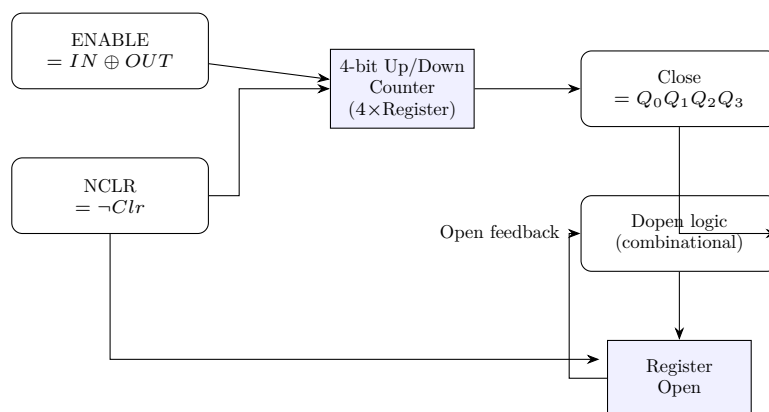
هدف آزمایش

- آشنایی عملی با طراحی یک مدار ترتیبی در Logisim: قطعه‌ی Register، پایه‌های rst/en/clk، و تفاوت رفتار reset ناهم‌گام (asynchronous) در برابر بارگذاری هم‌گام با کلاک.
- ساخت یک شمارنده‌ی بالا/پایین از قطعات پایه (نه از قطعه‌ی آماده‌ی Counter)، با درک زنجیره‌ی انتشار toggle-enable بین بیت‌ها (مشابه رقم نقلی در جمع‌کننده‌های زنجیره‌ای آزمایش اول، اما برای شرط تگل شدن هر بیت).
- طراحی یک مسیر فیدبک ترکیبی-ترتیبی: خروجی Open به‌عنوان یک قفل (latch) رفتار می‌کند که هم از راه یک شرط ورودی («ست») و هم از راه مقدار فعلی خودش (فیدبک) تغذیه می‌شود.
- تفکیک دقیق بین منطق ترکیبی محض (Dopen، Close) و بخش ترتیبی (۵ قطعه‌ی Register) در یک مدار واحد.

رابط ورودی/خروجی

سیگنال	جهت	پهنا	توضیح
Clk	ورودی	۱	کلاک سیستم
Clr	ورودی	۱	ریست ناهم‌گام، فعال در سطح پایین (Clr=0) یعنی ریست)
IN	ورودی	۱	پالس «یک نفر وارد شد» – هم شمارنده را بالا می‌برد، هم Open را خاموش می‌کند
OUT	ورودی	۱	پالس «یک نفر خارج شد» – شمارنده را پایین می‌آورد
Ent	ورودی	۱	دکمه‌ی درخواست ورود
T	ورودی	۱	۱ یعنی داخل بازه‌ی زمانی مجاز
Open	خروجی	۱	فرمان باز شدن در (سطح، نه پالس)
Close	خروجی	۱	۱ یعنی اتاق پر است (۱۵ نفر)
۳..Q۰Q	خروجی (اشکال‌زدایی)	۱ هرکدام	۴ بیت شمارنده‌ی جمعیت داخل اتاق
DBG EN	خروجی (اشکال‌زدایی)	۱ هرکدام	مشاهده‌ی مستقیم سیگنال‌های داخلی ENABLE، D0، U
•DBG D			
DBG U			

معماری کلی



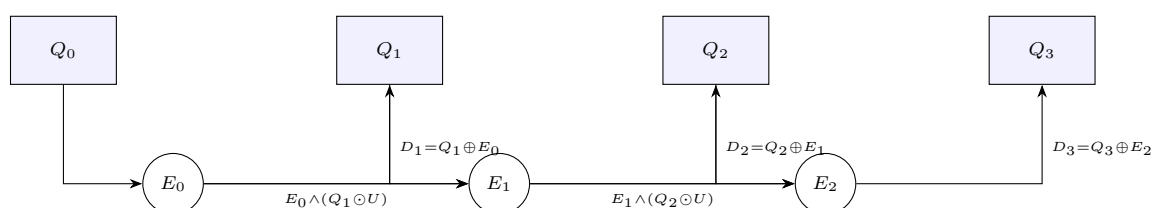
هیچ زیرمداری (subcircuit) در این پروژه نیست - برخلاف آزمایش اول که ساختار کاملاً سلسله‌مراتبی بود، اینجا همه چیز در یک مدار مسطح waiting_room پیاده شده و بلوک‌بندی فقط در برجسب تونل‌ها (ENABLE, NCLR, CLOSE, OPENQ, TOGGLE3, TOGGLE2, E0, D0..D3, Q0..Q3) دیده می‌شود.

شمارنده‌ی بالا/پایین ۴ بیتی

هر بیت شمارنده با یک Register تک‌بیتی ($\text{rst}=\text{NCLR}$, $\text{en}=\text{ENABLE}$, $\text{clk}=\text{Clk}$) پیاده شده. جهت شمارش با سیگنال $U \triangleq \text{IN}$ تعیین می‌شود: وقتی $U=1$ (کسی وارد می‌شود) شمارنده بالا می‌رود، وقتی $U=0$ (کسی خارج می‌شود، یعنی OUT پالس زده) پایین می‌آید. مثل رقم نقلی در جمع‌کننده‌ی زنجیره‌ای، اینجا هم یک سیگنال toggle-enable از بیت i به بیت $i+1$ منتقل می‌شود - بیت $i+1$ فقط زمانی نگل می‌کند که «تمام بیت‌های پایین‌تر با جهت شمارش U همسو باشند» (دقیقاً تعمیم قاعده‌ی معروف شمارنده‌ی باینری بالا/پایین):

$$\begin{aligned} D_0 &= \neg Q_0, & (\text{بیت صفر همیشه نگل می‌کند}) \\ E_0 &= Q_0 \odot U & (\text{XNOR: آیا بیت صفر با جهت شمارش همسوست؟}) \\ D_1 &= Q_1 \oplus E_0 \\ E_1 &= E_0 \wedge (Q_1 \odot U) \\ D_2 &= Q_2 \oplus E_1 \\ E_2 &= E_1 \wedge (Q_2 \odot U) \\ D_3 &= Q_3 \oplus E_2 \end{aligned}$$

هر $\odot$ (XNOR) با یک NOT+XOR ساخته شده (کتابخانه محدود به گیت‌های پایه است). جمع کل: ۳ گیت XOR برای محاسبه‌ی $Q_i \oplus U$ ، ۳ گیت NOT برای تبدیل به XNOR (E_0, E_1 ی بیت‌های ۱ و ۲، و پایه‌ی D_0)، ۲ گیت AND برای انتشار E_1, E_2 ، و ۳ گیت XOR برای محاسبه‌ی نهایی D_1, D_2, D_3 - در مجموع ۱۲ گیت برای کل منطق شمارنده (به‌علاوه‌ی ۴ Register). چون فقط وقتی ۱ است که همه‌ی بیت‌های $i \dots 0$ با U همسو باشند، این دقیقاً معادل «انتقال رقم نقلی» یک شمارنده‌ی باینری معمولی است، با این تفاوت که پیمانه‌ی مقایسه به‌جای «۱ ثابت» متغیر U است تا هم شمارش بالا و هم پایین با یک مدار پوشش داده شود.



منطق «اتاق پر است» (Close)

Close کاملاً ترکیبی است (بدون هیچ فلیپ‌فلاپ اضافه) و فقط از سه گیت AND دو ورودی زنجیره‌ای ساخته شده:

$$\text{Close} = Q_0 \wedge Q_1 \wedge Q_2 \wedge Q_3 \quad (15 = \text{یعنی شمارش})$$

از آنجا که این عبارت مستقیم از خروجی Registerهای شمارنده گرفته می‌شود (نه از یک نسخه‌ی تأخیری)، Close همیشه وضعیت لحظه‌ای اتاق را نشان می‌دهد — در همان سیکل کلاکی که شمارنده به ۱۵ می‌رسد، Close هم بی‌درنگ ۱ می‌شود.

منطق «باز شدن در» (Open)

بخش Open یک قفل SR گونه است: یک قطعه‌ی Register با ورودی D از یک شبکه‌ی ترکیبی که هم به یک شرط «ست» و هم به مقدار فعلی خود *Open* (فیدبک) وابسته است:

$$\begin{aligned} \text{LT15} &= \neg \text{Close} && (\text{هنوز جا هست}) \\ \text{EntValid} &= \text{LT15} \wedge \text{Ent} \wedge T && (\text{درخواست معتبر ورود}) \\ \text{orr} &= \text{EntValid} \vee \text{Open} && (\text{ست، یا از قبل باز بوده}) \\ \text{Dopen} &= \text{orr} \wedge \neg \text{IN} && (\text{اما اگر همین حالا کسی رد شد، ببند}) \end{aligned}$$

Register این بخش با $\text{clk} = \text{Clk}$ و $\text{en} = 1$ (یک Constant ثابت — یعنی همیشه در هر لبه‌ی بالارونده نمونه‌برداری می‌کند) و $\text{rst} = \text{NCLR}$ پیکربندی شده. رفتار حاصل: به محض یک درخواست معتبر (EntValid)، Open در لبه‌ی بعدی کلاک ست می‌شود و تا وقتی پالس IN نیامده (یعنی شخص واقعاً از در رد نشده) از راه فیدبک orr روشن می‌ماند — حتی اگر Ent زودتر رها شود. همین‌که $\text{IN} = 1$ شود، عبارت $\neg \text{IN}$ صفر می‌شود و Dopen در همان لبه صفر می‌گردد، یعنی Open در لبه‌ی بعدی خاموش می‌شود. توجه شود که این خاموش شدن با همان پالس IN است که شمارنده را هم بالا می‌برد — یعنی «عبور شخص از در» دقیقاً همان رویدادی است که هم شمارش را به‌روز و هم قفل در را آزاد می‌کند.

ریست ناهم‌گام

پایه‌ی Clr در کل مدار فقط یک بار معکوس می‌شود ($\text{Clr} = \neg \text{NCLR}$) و این یک سیگنال به پایه‌ی rst هر پنج Register (۴ بیت شمارنده + Open) وصل است. چون یک Register در Logisim ناهم‌گام (asynchronous) عمل می‌کند، وقتی $\text{Clr} = 0$ می‌شود همه‌ی حالت‌ها (شمارش و Open) فوراً صفر می‌شوند — مستقل از سطح یا لبه‌ی کلاک — دقیقاً همان رفتاری که سناریوی reset_mid_operation در تست‌ها بررسی می‌کند.

ردیابی دستی (برای ارائه)

مثال ۱ - شروع از صفر، سه ورود پیاپی (بخشی از سناریوی up_counting_full؛ در همه‌ی گام‌ها T=1):

رویداد	ورودی	ENABLE	Dopen	$Q_3Q_2Q_1Q_0$ بعد	Open بعد
ریست	$\bullet \text{Clr} =$	—	—	۰۰۰۰	۰
Ent فشرده می‌شود	$\neg \text{Ent} =$	$\bullet \oplus \bullet = \bullet$	$\neg \text{EntValid} \rightarrow$ $\neg \text{EntValid} \Rightarrow \neg \text{IN} = 1$	۰۰۰۰ (تغییر)	۱ (بدون)
شخص رد می‌شود	$\neg \text{IN} =$	$1 \oplus \bullet = 1$	$\neg \text{EntValid} \rightarrow$ $\neg \text{EntValid} \Rightarrow \neg \text{IN} = \bullet$	۰۰۰۱	۰
Ent دوباره	$\neg \text{Ent} =$	$\bullet \oplus \bullet = \bullet$	مشابه گام دوم $1 \Rightarrow$	۰۰۰۱ (تغییر)	۱ (بدون)
شخص رد می‌شود	$\neg \text{IN} =$	$1 \oplus \bullet = 1$	مشابه گام سوم $\bullet \Rightarrow$	۰۰۱۰	۰

مثال ۲ - اتاق پر، سپس یک نفر خارج می‌شود (بخشی از سناریوی exit_and_reentry؛ شمارنده روی ۱۵ است):

رویداد	ورودی	وضعیت پیش از لبه	وضعیت پس از لبه
درخواست ورود با اتاق پر	$\neg \text{Ent} =$ $\bullet \text{IN} =$ $\bullet \text{OUT} =$	$Q = 1111$ $\bullet = \text{EntValid} \Rightarrow \bullet = \text{LT15}$	$\bullet = \text{Open} \Rightarrow$ $(\bullet = \bullet \vee \bullet = \text{orr})$
یک نفر خارج می‌شود	$\bullet \text{IN} =$ $\neg \text{OUT} =$	$Q = 1111$ پایین	$E \bullet = Q \bullet \oplus \bullet = 1 \oplus \bullet = \bullet \Rightarrow$ زنجیره $D \bullet = \neg Q \bullet = \bullet$ $\bullet = \text{Close} \Rightarrow Q = 1110 = 14$
حالا درخواست ورود پذیرفته می‌شود	$\neg \text{Ent} =$ $\bullet \text{IN} =$ $\bullet \text{OUT} =$	$Q = 1110$ $\neg \text{EntValid} \Rightarrow \bullet = \text{LT15}$	$\bullet = \text{Open} \Rightarrow$

چیدمان و سیم‌کشی

- مدار مسطح است (بدون زیرمدار)؛ به جای آن، هر سیگنال به اشتراک رفته بین بخش‌های دوردست مدار (ENABLE، Tunnel هم‌نام عبور می‌کند - دقیقاً همان تکنیک «سیم بی‌نام‌نویسی راه دور» که در آزمایش اول برای جلوگیری از تداخل بصری معرفی شد، اینجا هم برای اتصال چهار Register پراکنده در صفحه (شمارنده) و شبکه‌ی منطق Open در سمت راست صفحه به کار رفته.
- چهار پایه‌ی خروجی اشکال‌زدایی Q0..Q3 (علاوه بر پایه‌های اصلی Close/Open) و سه پایه‌ی DBG_U/DBG_D0/DBG_EN برای مشاهده‌ی مستقیم سیگنال‌های داخلی حین شبیه‌سازی گذاشته شده‌اند - بدون این‌ها، عیب‌یابی زنجیره‌ی toggle-enable فقط از روی خروجی Close تقریباً غیرممکن بود.
- برجسب‌های تونل تکراری (مثلاً چند تونل جدا با برجسب Q0) در نقاط مختلف صفحه گذاشته شده‌اند تا سیم‌های بلند از وسط گیت‌های دیگر رد نشوند - همه‌شان الکتریکی یک شبکه‌اند، صرفاً محل ورودشان به مدار فرق دارد.

نتایج تست

مدل مرجع رفتار مدار (gen_tests.py) به‌طور مستقل از خود مدار پیاده‌سازی شده و روی ۸ سناریوی هدفمند (نه فقط ورودی تصادفی) بردار تست تولید می‌کند؛ همه‌ی بردارها با اجرای واقعی Logisim در حالت headless (-test- vector) روی خود waiting_room.circ تأیید شده‌اند:

ردیف	چه چیزی را بررسی می‌کند	سناریو
۷	ریست اولیه، سکون بدون رویداد	reset_and_basic
۶۳	شمارش کامل ۰ تا ۱۵ با Ent، سپس رد شدن ورودی ۱۶م	up_counting_full
۷	Ent در T=0 باید همیشه رد شود	out_of_hours_blocked
۶۹	پر شدن اتاق، خروج یک نفر، ورود دوباره	exit_and_reentry
۹	ورود و خروج هم‌زمان $\Rightarrow$ شمارش ثابت می‌ماند	simultaneous_in_out
۱۱	Open تا رسیدن پالس IN قفل می‌ماند	open_holds_until_in
۲۱	شمارش پایین‌رونده‌ی کامل تا صفر	down_counting
۱۴	ریست نهم‌گام وسط عملیات	reset_mid_operation
۲۰۱	جمع (tests_waiting_room.vec)	

علاوه بر این، فایل debug_counter.vec (۶ ردیف) به‌صورت جداگانه فقط منطق شمارش رو به بالا را روی چند سیکل اول بررسی می‌کند. خروجی واقعی اجرای run_tests.sh:

```

circ : waiting_room.circ
java : /opt/homebrew/opt/openjdk@21/bin/java
jar   : /Applications/Logisim-evolution.app/Contents/app/logisim-evolution-4.1.0-all.jar
vecs : 2

=== debug_counter.vec → circuit 'waiting_room' ===
Loading test vector "debug_counter.vec"...
Running 6 vectors...
Passed: 6, Failed: 0
PASS: debug_counter.vec

=== tests_waiting_room.vec → circuit 'waiting_room' ===
Loading test vector "tests_waiting_room.vec"...
Running 201 vectors...
Passed: 201, Failed: 0
PASS: tests_waiting_room.vec

summary: 2/2 passed, 0 failed

```

جمع کل: ۲ از ۲ مجموعه‌تست پاس (۲۰۷ ردیف بردار تست)، صفر شکست.

نکات پیاده‌سازی

- اعتبارسنجی مقدار مجاز صفت size: Logisim-evolution برای صفت size (اندازه‌ی ظاهری گیت) دو مجموعه‌ی مقدار مجاز جدا دارد — گیت‌های XNOR/XOR/NOR/NAND/OR/AND {۳۰, ۵۰, ۷۰} را می‌پذیرند، اما NOT Gate فقط {۲۰, ۳۰} را. مقداردهی "size=50" به یک NOT Gate (که ظاهراً از کپی‌کردن تنظیمات گیت‌های دیگر پیش آمده بود) باعث خطای بارگذاری (50 is not a valid attribute value) می‌شد؛ این مقدار برای هر ۷ نمونه‌ی NOT Gate در فایل به ۳۰ اصلاح شد.
- نوار ابزار خالی: عنصر <toolbar/> فایل به‌صورت خالی (بدون هیچ <tool>) ذخیره شده بود که باعث می‌شد Logisim نوار ابزار خالی نشان دهد؛ نوار ابزار با مجموعه‌ی استاندارد ابزارها (Text/Wiring/Edit/Poke)، Pin ورودی و خروجی، Tunnel، Constant، گیت‌های پایه، Register، Counter) پر شد.

- تأیید با اجرای واقعی Logisim، نه فقط تجزیه‌ی XML: هر دو اصلاح بالا، هم با تجزیه‌ی فایل و هم با اجرای واقعی Logisim-evolution 4.1.0 در حالت tty stats (که همه‌ی ۱۱۲ قطعه را بدون خطا می‌شمارد) و در حالت test-vector (هر ۲۰۷ ردیف بردار تست) دوباره بررسی شدند.